LoanDesk — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── data/
│   └── sample/
│       └── tasks.json
├── scripts/
│   └── compare_with_tools.py
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── verifier.py
│   │   └── worker.py
│   ├── chains/
│   │   ├── __init__.py
│   │   └── direct_answer.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   ├── direct.txt
│   │   ├── verifier.txt
│   │   └── worker.txt
│   ├── services/
│   │   ├── __init__.py
│   │   └── finance_service.py
│   ├── tools/
│   │   ├── __init__.py
│   │   ├── calculator.py
│   │   ├── fx.py
│   │   └── policy_search.py
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── http_client.py
│   │   ├── logger.py
```

```
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   └── schemas.py
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py
│   └── unit/
│       ├── test_agents.py
│       ├── test_config.py
│       └── test_tools.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
└── requirements.txt
```

Setup files

`requirements.txt`

```
langchain-core>=0.3
langchain-openai>=0.2
pydantic>=2.0
python-dotenv>=1.0
requests>=2.31
```

`.env.example`

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
```

`.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
```

Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text with your name and year.

Settings, errors, logging, prompts

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass


class TransientHTTPError(Exception):
    """A retryable HTTP failure (timeout, 429, 5xx)."""
    pass


class PermanentHTTPError(Exception):
    """A non-retryable HTTP failure (most 4xx)."""
    pass


class MaxIterationsExceeded(Exception):
    """The Worker ran more steps than its hard limit."""
    def __init__(self, message, steps):
        super().__init__(message)
        self.steps = steps
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
    log_level: str


def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value
```

```
def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os


def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py
PROMPTS_DIR = "src/prompts"


def load_prompt(name: str) -> str:
    with open(PROMPTS_DIR + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/utils/http_client.py
# Doc02's retrying HTTP client: timeout on every call, retry timeouts/429/5xx
# with a backoff, raise PermanentHTTPError on other 4xx.
import time

import requests

from src.exceptions import PermanentHTTPError, TransientHTTPError
from src.utils.logger import get_logger
```

```
RETRY_STATUS = (429, 500, 502, 503, 504)


def compute_backoff_delay(attempt: int) -> float:
    return min(2 ** attempt, 8)


def request_with_retry(method, url, max_attempts=5, **kwargs):
    logger = get_logger(__name__)
    kwargs.setdefault("timeout", 10)
    for attempt in range(max_attempts):
        try:
            response = requests.request(method, url, **kwargs)
        except requests.Timeout:
            logger.warning(f"timeout on attempt {attempt}")
            time.sleep(compute_backoff_delay(attempt))
            continue
        if response.status_code < 400:
            return response.json()
        if response.status_code in RETRY_STATUS:
            logger.warning(f"status {response.status_code}, retrying")
            time.sleep(compute_backoff_delay(attempt))
            continue
        raise PermanentHTTPError(f"HTTP {response.status_code} for {url}")
    raise TransientHTTPError(f"still failing after {max_attempts} attempts")
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

MODEL_NAME = "gpt-4o-mini"


def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model=MODEL_NAME, temperature=0, api_key=config.openai_api_key
    )
```

The direct chain (the baseline that only guesses)

```
# src/prompts/direct.txt
Answer the question as best you can from your own knowledge.
```

```
Question: {question}
```

```
# src/chains/direct_answer.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.utils.prompts import load_prompt


def build_direct_chain(model):
    prompt = ChatPromptTemplate.from_template(load_prompt("direct"))
    return prompt | model | StrOutputParser()
```

The three tools

```
# src/tools/calculator.py
# Pure-logic tool: the exact amortized monthly loan payment. No network.
from langchain_core.tools import tool


@tool
def monthly_payment(principal: float, annual_rate_percent: float,
                    months: int) -> float:
    """Exact monthly payment for an amortized loan. Use for any 'what is
    the monthly payment' question, never estimate it."""
    monthly_rate = annual_rate_percent / 100 / 12
    if monthly_rate == 0:
        return round(principal / months, 2)
    factor = (1 + monthly_rate) ** months
    payment = principal * monthly_rate * factor / (factor - 1)
    return round(payment, 2)
```

```
# src/tools/fx.py
# Live-API tool: today's exchange rate from the keyless frankfurter.app
# service, via Doc02's retrying client.
from langchain_core.tools import tool

from src.utils.http_client import request_with_retry

FX_URL = "https://api.frankfurter.app/latest"
```

```
@tool
def get_exchange_rate(amount: float, base: str, quote: str) -> str:
    """Convert an amount between two currencies at today's real rate. Use
    for any currency conversion, never guess the rate."""
    data = request_with_retry(
        "GET", FX_URL, params={"from": base.upper(), "to": quote.upper()}
    )
    rate = data["rates"][quote.upper()]
    converted = round(amount * rate, 2)
    label = f"{amount} {base.upper()} = {converted} {quote.upper()}"
    return f"{label} (rate {rate})"
```

```
# src/tools/policy_search.py
# A policy lookup that fails on its FIRST call on purpose, so every run
# proves the Worker recovers from a tool failure instead of crashing.
from langchain_core.tools import tool

POLICIES = {
    "early repayment": "Early repayment is allowed with no fee after 6 "
                       "months; before that a 1% fee applies.",
    "late payment": "A late payment fee of 2% applies after a 10-day grace "
                    "period.",
}


class FlakyToolState:
    call_count = 0


@tool
def search_policies(query: str) -> str:
    """Search loan policies by keyword and return the matching policy text.
    Use for any question about rules, fees, or what is allowed."""
    FlakyToolState.call_count += 1
    if FlakyToolState.call_count == 1:
        raise ConnectionError("policy service timed out")
    for key, text in POLICIES.items():
        if key in query.lower():
            return text
    return "No matching policy found."
```

The Worker (the hand-built loop)

```
# src/prompts/worker.txt
You are LoanDesk, a careful finance-desk assistant. You have tools:
- monthly_payment: for any loan monthly-payment calculation.
- get_exchange_rate: for any currency conversion (uses today's real rate).
- search_policies: for any question about rules, fees, or what is allowed.

Use the right tool for the question; do not guess numbers or rates. If a
tool fails, try it once more. Answer ONLY from what the tools returned.
For general questions that need no tool, answer directly.
```

```
# src/agents/worker.py
# The Worker: a hand-built think -> act -> observe loop over the tools, with
# a step log and a hard step limit. No create_agent/AgentExecutor.
from dataclasses import dataclass

from langchain_core.messages import (
    HumanMessage,
    SystemMessage,
    ToolMessage,
)

from src.exceptions import MaxIterationsExceeded
from src.tools.calculator import monthly_payment
from src.tools.fx import get_exchange_rate
from src.tools.policy_search import search_policies
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

ALL_TOOLS = [monthly_payment, get_exchange_rate, search_policies]


@dataclass
class AgentResult:
    final_answer: str
    steps: list


def find_tool(tools, name):
    for tool in tools:
        if tool.name == name:
            return tool
    return None


def run_tool(call, tools):
    tool = find_tool(tools, call["name"])
```

```
    if tool is None:
        return None, f"Error: unknown tool {call['name']}"
    try:
        result = tool.invoke(call["args"])
    except Exception as e:
        return None, f"Error: {e}"
    return str(result), None


def run_worker(model, task, max_iterations=6):
    logger = get_logger(__name__)
    model_with_tools = model.bind_tools(ALL_TOOLS)
    messages = [
        SystemMessage(content=load_prompt("worker")),
        HumanMessage(content=task),
    ]
    steps = []

    for _ in range(max_iterations):
        reply = model_with_tools.invoke(messages)
        messages.append(reply)
        if not reply.tool_calls:
            return AgentResult(final_answer=reply.content, steps=steps)
        for call in reply.tool_calls:
            result, error = run_tool(call, ALL_TOOLS)
            if error is None:
                observation = result
            else:
                observation = error
            logger.info(f"tool {call['name']} -> {observation[:40]}")
            messages.append(ToolMessage(
                content=observation, tool_call_id=call["id"]
            ))
            steps.append({"tool": call["name"], "args": call["args"],
                          "result": observation})

    raise MaxIterationsExceeded("worker did not finish in time", steps)
```

The Verifier

```
# src/schemas.py
from pydantic import BaseModel


class VerifierVerdict(BaseModel):
```

```
    approved: bool
    reason: str
```

```
# src/prompts/verifier.txt
You check a finance answer against the evidence the tools actually
returned. Approve ONLY if the answer is fully supported by the evidence.
If the answer states a number or fact not in the evidence, reject it.

Question:
{question}

Tool evidence (the only source of truth):
{evidence}

Answer to check:
{answer}
```

```
# src/agents/verifier.py
# The Verifier: a second agent with its own prompt that checks the Worker's
# answer against only the SUCCESSFUL tool results.
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import VerifierVerdict
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt


def build_verifier_chain(model):
    prompt = ChatPromptTemplate.from_template(load_prompt("verifier"))
    return prompt | model.with_structured_output(VerifierVerdict)


def collect_evidence(steps) -> str:
    lines = []
    for step in steps:
        if not str(step["result"]).startswith("Error"):
            lines.append(f"{step['tool']}: {step['result']}")
    return "\n".join(lines)


def verify(chain, question, result):
    logger = get_logger(__name__)
    evidence = collect_evidence(result.steps)
    verdict = chain.invoke({
        "question": question,
```

```
        "evidence": evidence,
        "answer": result.final_answer,
    })
    logger.info(f"verifier approved={verdict.approved}")
    return verdict
```

The service (Worker → Verifier → one retry)

```
# src/services/finance_service.py
# The handoff: run the Worker, verify; if rejected, add the reason and run
# the Worker ONCE more, then verify again against the original question.
from src.agents.verifier import verify
from src.agents.worker import run_worker
from src.utils.logger import get_logger


def answer_task(model, verifier_chain, task, max_iterations=6):
    logger = get_logger(__name__)
    result = run_worker(model, task, max_iterations)
    verdict = verify(verifier_chain, task, result)
    if not verdict.approved:
        logger.info("rejected, running one retry")
        retry_task = (
            f"{task}\n\nA reviewer rejected a previous answer for this "
            f"reason: {verdict.reason}\nAnswer again, using the tools."
        )
        result = run_worker(model, retry_task, max_iterations)
        verdict = verify(verifier_chain, task, result)
    return result, verdict
```

The entry point

```
# main.py
import sys

import openai

from src.agents.verifier import build_verifier_chain
from src.config import load_config
from src.exceptions import MaxIterationsExceeded, MissingConfigError
from src.models.llm import get_model
from src.services.finance_service import answer_task
```

```
def print_steps(steps):
    for step in steps:
        print(f"  -> {step['tool']} {step['args']} => {step['result']}")


def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    model = get_model(config)
    verifier_chain = build_verifier_chain(model)
    print("LoanDesk — type a finance question, or /quit to exit.")
    while True:
        task = input("You: ").strip()
        if task == "/quit":
            break
        if task == "":
            continue
        try:
            result, verdict = answer_task(model, verifier_chain, task)
        except MaxIterationsExceeded as e:
            print("Could not finish in time. Steps so far:")
            print_steps(e.steps)
            continue
        except openai.AuthenticationError:
            print("Login failed — check OPENAI_API_KEY in .env.")
            sys.exit(1)
        except openai.RateLimitError:
            print("Busy right now — try again in a moment.")
            continue
        print_steps(result.steps)
        mark = "verified" if verdict.approved else "NOT verified"
        print(f"Answer ({mark}): {result.final_answer}")


if __name__ == "__main__":
    main()
```

The compare script and its data

```
# data/sample/tasks.json
{
  "tasks": [
    "What is the monthly payment on a 20000 loan at 7% annual for 36 months?",
    "What is 500 EUR in USD today?",
    "Does our policy allow early repayment without a fee?"
  ]
}
```

```
# scripts/compare_with_tools.py
# Shows the project's whole point: a memory guess next to a checked answer.
# Run from the repo root: python -m scripts.compare_with_tools
import json

from src.agents.verifier import build_verifier_chain
from src.chains.direct_answer import build_direct_chain
from src.config import load_config
from src.models.llm import get_model
from src.services.finance_service import answer_task

TASKS_PATH = "data/sample/tasks.json"


def main():
    config = load_config()
    model = get_model(config)
    direct = build_direct_chain(model)
    verifier_chain = build_verifier_chain(model)
    with open(TASKS_PATH) as tasks_file:
        tasks = json.loads(tasks_file.read())["tasks"]
    for task in tasks:
        print(f"\nTask: {task}")
        print(f"  guess:   {direct.invoke({'question': task})}")
        result, verdict = answer_task(model, verifier_chain, task)
        print(f"  checked: {result.final_answer} (approved={verdict.approved})")


if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "LOANDESK_TEST_SETTING"


def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_empty_setting() -> None:
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")


def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")


if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_tools.py
# The tools directly, no model. The FX tool is not called here (it needs the
```

```
# network); its integration test covers it. Run from the repo root:
#   python -m tests.unit.test_tools
from pydantic import ValidationError

from src.tools.calculator import monthly_payment
from src.tools.policy_search import FlakyToolState, search_policies


def test_payment_math() -> None:
    value = monthly_payment.invoke(
        {"principal": 20000, "annual_rate_percent": 7, "months": 36}
    )
    result = "OK"
    if round(value, 2) != 617.54:
        result = "WRONG"
    print(f"case 1 (20000 @7% /36 = 617.54) -> {result}")


def test_payment_rejects_bad_arg() -> None:
    result = "WRONG"
    try:
        monthly_payment.invoke(
            {"principal": "lots", "annual_rate_percent": 7, "months": 36}
        )
    except ValidationError:
        result = "OK"
    print(f"case 2 (bad principal refused)  -> {result}")


def test_policy_search_recovers() -> None:
    FlakyToolState.call_count = 0
    result = "OK"
    try:
        search_policies.invoke({"query": "early repayment"})
    except ConnectionError:
        pass
    else:
        result = "WRONG"
    text = search_policies.invoke({"query": "early repayment"})
    if "Early repayment" not in text:
        result = "WRONG"
    print(f"case 3 (fails once, then works) -> {result}")


if __name__ == "__main__":
    test_payment_math()
```

```
    test_payment_rejects_bad_arg()
    test_policy_search_recovers()
```

```
# tests/unit/test_agents.py
# The Worker/Verifier helpers on hand-made data — no model.
# Run from the repo root: python -m tests.unit.test_agents
from src.agents.verifier import collect_evidence
from src.agents.worker import ALL_TOOLS, find_tool, run_tool


def test_find_known_tool() -> None:
    result = "OK"
    if find_tool(ALL_TOOLS, "monthly_payment") is None:
        result = "WRONG"
    print(f"case 1 (known tool found)      -> {result}")


def test_find_unknown_tool() -> None:
    result = "OK"
    if find_tool(ALL_TOOLS, "send_money") is not None:
        result = "WRONG"
    print(f"case 2 (unknown tool is None)  -> {result}")


def test_run_tool_good_call() -> None:
    call = {"name": "monthly_payment", "id": "1",
            "args": {"principal": 1200, "annual_rate_percent": 0,
                     "months": 12}}
    value, error = run_tool(call, ALL_TOOLS)
    result = "OK"
    if error is not None or value != "100.0":
        result = "WRONG"
    print(f"case 3 (good call -> result)   -> {result}")


def test_run_tool_unknown() -> None:
    call = {"name": "send_money", "id": "1", "args": {}}
    value, error = run_tool(call, ALL_TOOLS)
    result = "OK"
    if value is not None or error is None:
        result = "WRONG"
    print(f"case 4 (unknown -> error)      -> {result}")


def test_collect_evidence_drops_errors() -> None:
    steps = [
```

```
        {"tool": "a", "args": {}, "result": "Error: boom"},
        {"tool": "b", "args": {}, "result": "good data"},
    ]
    evidence = collect_evidence(steps)
    result = "OK"
    if "Error" in evidence or "good data" not in evidence:
        result = "WRONG"
    print(f"case 5 (evidence drops errors) -> {result}")


if __name__ == "__main__":
    test_find_known_tool()
    test_find_unknown_tool()
    test_run_tool_good_call()
    test_run_tool_unknown()
    test_collect_evidence_drops_errors()
```

```
# tests/integration/test_live_agents.py
# The parts that need the real model (and the FX API). Run from the repo
# root: python -m tests.integration.test_live_agents
from src.agents.verifier import build_verifier_chain, verify
from src.agents.worker import run_worker
from src.config import load_config
from src.models.llm import get_model
from src.services.finance_service import answer_task


def tools_used(result):
    names = []
    for step in result.steps:
        names.append(step["tool"])
    return names


def test_payment_uses_calculator(model) -> None:
    result = run_worker(
        model, "Monthly payment on 20000 at 7% annual over 36 months?"
    )
    ok = "monthly_payment" in tools_used(result)
    print(f"case 1 (calc used)          -> {'OK' if ok else 'WRONG'}")


def test_fx_uses_rate_tool(model) -> None:
    result = run_worker(model, "What is 500 EUR in USD today?")
    ok = "get_exchange_rate" in tools_used(result)
    print(f"case 2 (fx tool used)       -> {'OK' if ok else 'WRONG'}")
```

```
def test_policy_recovers(model) -> None:
    result = run_worker(model, "Does policy allow early repayment fee-free?")
    ok = "search_policies" in tools_used(result)
    print(f"case 3 (policy recovered)   -> {'OK' if ok else 'WRONG'}")


def test_verifier_rejects_wrong(model) -> None:
    chain = build_verifier_chain(model)
    result = run_worker(model, "What is 500 EUR in USD today?")
    # tamper with the answer so it no longer matches the evidence
    result.final_answer = "500 EUR is 999999 USD."
    verdict = verify(chain, "What is 500 EUR in USD today?", result)
    print(f"case 4 (wrong answer rejected) -> "
          f"{'OK' if not verdict.approved else 'WRONG'}")


def test_full_handoff(model) -> None:
    chain = build_verifier_chain(model)
    result, verdict = answer_task(
        model, chain, "Monthly payment on 10000 at 5% over 24 months?"
    )
    ok = verdict.approved and "monthly_payment" in tools_used(result)
    print(f"case 5 (full handoff approves) -> {'OK' if ok else 'WRONG'}")


if __name__ == "__main__":
    config = load_config()
    model = get_model(config)
    test_payment_uses_calculator(model)
    test_fx_uses_rate_tool(model)
    test_policy_recovers(model)
    test_verifier_rejects_wrong(model)
    test_full_handoff(model)
```

Expected unit output (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (20000 @7% /36 = 617.54) -> OK
case 2 (bad principal refused)  -> OK
case 3 (fails once, then works) -> OK
```

```
case 1 (known tool found)      -> OK
case 2 (unknown tool is None)  -> OK
case 3 (good call -> result)   -> OK
case 4 (unknown -> error)      -> OK
case 5 (evidence drops errors) -> OK
```

That's the whole project. If your Worker calls the calculator for maths, the FX tool for a rate, the policy search for a rule (recovering from its first-call failure), never loops forever, and your Verifier approves right answers while rejecting wrong ones — you've built it.